

개략적인 동작은 각 레포트에서 작성했으니 이론적인 부분만 적어보려고 합니다.

fork()를 사용하는 방법에서는 각 클라이언트 연결마다 새로운 프로세스를 생성합니다. 부모 프로세스의 주소공간을 복사한 새로운 자식이 프로세스를 생성합니다. 메모리의 경우도 부모 프로세스와 자식 프로세스 간에 격리가 되어 있습니다. 한 프로세스의 변경 사항은 대체로 다른 프로세스에 영향을 미치지 않는 것을 알 수 있습니다.

자식 프로세스가 부모 프로세스의 변수를 복사하더라도 이후의 변경에는 다른 프로세스에 영향을 주지 않는다는 것 또한 **fork()**의 핵심이라고 할 수 있습니다. 결국 프로세스간 분리가 되어있어 하나의 프로세스가 충돌해도 다른 프로세스에 영향을 미치지 않는 안정성을 발휘할 수 있습니다. 다만 프로세스를 생성하고, 프로세스 단위로 동작하므로 오버헤드는 상당하다는 것을 알 수 있습니다.

반대로 **Pthread**를 사용하는 방식은 **POSIX** 스레드를 사용하는 방식으로 스레드의 생성과 메모리 관리의 과정에서 전역변수와 힙에 할당된 메모리는 모든 스레드가 공유하고 스택변수는 스레드마다 독립적으로 점유하고 있습니다..

스레드는 동일한 프로세스 내부에서 실행하므로 생성 및 종료 오버헤드가 적지만 안정성은 떨어질 수 있다는 점이 있습니다. 스레드의 경우 프로세스와 다르게 다양한 동기화 작업을 지원하는 함수(라이브러리)(**mutex**, **semaphore**)가 존재하고 각 스레드간 통신이 프로세스 보다 쉽습니다.

특징	fork()	pthread.h
프로세스/스레드 생성	각 클라이언트 연결마다 새로운 프로세스를 생성	각 클라이언트 연결마다 새로운 스레드를 생성
메모리 관리	프로세스 간 메모리 격리, 독립적인 메모리 공간 사용	스레드 간 메모리 공유, 동일한 메모리 공간 사용
변수 관리	프로세스별 독립적인 변수 사용	전역 변수와 힙 메모리는 공유, 스택 변수는 독립적으로 사용
안정성	하나의 프로세스 충돌 시 다른 프로세스에 영향 없음	하나의 스레드 충돌 시 전체 프로세스에 영향
오버헤드	프로세스 생성 및 종료 오버헤드가 큼	스레드 생성 및 종료 오버헤드가 적음
메모리 사용량	독립적인 메모리 공간 사용으로 메모리 사용량 증가	메모리 공유로 메모리 사용량 절약
통신 비용	프로세스 간 통신에 IPC 필요, 추가 오버헤드 발생	스레드 간 통신에 IPC 불필요, 데이터 교환 용이
동기화 문제	프로세스 간 동기화 문제 없음	스레드 간 동기화 문제 발생 가능, mutex , 세마포어 등 필요
보안	프로세스 간 메모리 접근 불가, 보안성 높음	동일한 메모리 접근 가능, 보안성 낮음
효율성	자원 소모 큼, 비효율적	자원 소모 적음, 효율적